

TERRARIO

Un ecosistema en un frasco

Landing page estática — Proyecto académico

Presentación del proyecto: propósito, tecnología, arquitectura y aprendizajes



Propósito y objetivos del proyecto



Propósito

Diseñar una landing page comercial para "Terrario", una marca de terrarios cerrados y autosuficientes, que comunique su propuesta de valor y capture clientes potenciales sin depender de infraestructura de servidor.

01

Presentar el producto

Explicar qué es un terrario cerrado y por qué no requiere riego ni mantenimiento.

02

Generar confianza

Mostrar testimonios reales y fichas de cuidado por especie.

03

Facilitar la conversión

Exhibir planes y precios claros, con llamados a la acción directos.

04

Capturar leads

Ofrecer un formulario de contacto validado para recibir consultas.

Descripción de la problemática que resuelve

El problema

- Muchas personas quieren tener plantas en casa u oficina, pero desisten por miedo a no saber cuidarlas (riego, luz, plagas).
- Las marcas de plantas suelen vender el producto sin explicar el porqué funciona, generando desconfianza en la compra online.
- Construir una tienda con backend completo (Laravel, base de datos, hosting) es costoso y lento para validar una idea de negocio.

La solución del proyecto

- Una landing 100% estática (HTML/CSS/JS puro) que explica el concepto "ecosistema cerrado" con lenguaje simple y visual.
- Incluye fichas de cuidado, testimonios de clientes reales y soporte por foto, reduciendo la incertidumbre del comprador.
- Al no requerir servidor ni build step, se despliega en minutos en Vercel, ideal para validar el modelo de negocio a bajo costo.

Tecnologías utilizadas



HTML5

Estructura semántica del sitio (index.html): header, secciones ancladas, formulario y footer.



CSS3 modular

base.css y main.css orquestan componentes independientes en css/components/ (header, hero, cards, pricing, etc.).



JavaScript vanilla

terrario.js maneja el menú móvil, el año dinámico del footer y la validación del formulario, sin frameworks.



Google Fonts

Tipografías Fraunces, Work Sans e IBM Plex Mono cargadas vía CDN para dar identidad editorial a la marca.



Sin build step

No usa Laravel, bundlers ni backend: el sitio se sirve tal cual, minimizando dependencias.



Despliegue en Vercel

Al ser 100% estático, se publica directamente como hosting estático en minutos.

Arquitectura del sistema

Sitio estático de una sola página (SPA sin framework), organizado por responsabilidad de archivo:

Estructura de archivos

```
index.html
css/
  base.css
  main.css
  components/
    site-header.css
    hero.css
    feature-card.css
    testimonial.css
    pricing.css
    contact-form.css
    notification.css
    site-footer.css
js/
  terrario.js
```

1 Presentación (HTML)

index.html define header, hero, especies, testimonios, precios, contacto y footer.

2 Estilos (CSS por componente)

main.css importa base.css y cada archivo de css/components/, uno por bloque visual.

3 Comportamiento (JS)

terrario.js escucha DOMContentLoaded y agrega: toggle de menú, año automático y validación de formulario.

4 Interacción con el usuario

Al enviar el formulario válido, se muestra una notificación simulada (sin backend real).

Conclusiones y aprendizajes

Simplicidad efectiva

Un sitio 100% estático es suficiente para validar una idea de negocio antes de invertir en backend.

CSS modular escala mejor

Separar cada componente en su propio archivo facilita mantenimiento y localizar estilos rápidamente.

JS mínimo, impacto real

Con pocas líneas de JavaScript vanilla se logró navegación móvil, validación de formulario y feedback al usuario.

Accesibilidad desde el diseño

Uso de atributos aria-label y aria-expanded muestra que la accesibilidad se puede integrar sin frameworks.

Límite del enfoque estático

El formulario de contacto es una simulación; el siguiente paso sería integrarlo con un servicio real de envío de correos o backend ligero.

Despliegue instantáneo

Sin build step, el proyecto se publica en Vercel en minutos, ideal para iteración rápida.